GUIA DE EXECUÇÃO

Trabalho 1 – Avaliação de Desempenho de Cliente-Servidor de Arquivos

UC Redes de Computadores • ICT/UNIFESP

Escopo	Definição
Sistema operacional alvo	Windows 10 2004+ ou Windows 11, usando WSL2/Ubuntu
Tecnologias centrais	C, API de sockets, CORE, Docker Desktop, iperf3, ping/traceroute
Entrega do enunciado	Código(s) do servidor + vídeo-relatório de até 10 minutos
Prazo informado	05/11/2026
Tamanho do grupo	No máximo 3 alunos

Objetivo deste guia

Transformar o enunciado em um roteiro operacional: instalar o ambiente em Windows, implementar e testar o cliente-servidor em C, montar a topologia no CORE, executar uma campanha reprodutível, analisar os dados e preparar a entrega.

Como usar este guia

Siga as etapas na ordem. Cada etapa termina com uma verificação objetiva. Não avance apenas porque um comando terminou sem mensagem de erro: registre versões, endereços IP, parâmetros de rede, resultados de ping/iperf3 e os arquivos gerados. O resultado final deve permitir que outro integrante reproduza a execução a partir de um computador Windows limpo.

A rota principal deste documento usa WSL2, Ubuntu e Docker Desktop. O enunciado original apresenta uma rota com Vagrant, VirtualBox e uma VM Ubuntu Bionic; essa rota é mantida na Seção 4 como alternativa compatível, mas a rota principal é preferível porque o CORE atual documenta Ubuntu 22.04 em contêiner e exige Python 3.10+ na instalação nativa. [1] [2]

1. Requisitos do enunciado, sem omissões

Item	Requisito que precisa aparecer no trabalho	Evidência recomendada
Grupo e prazo	Grupo de até 3 alunos; entrega em 05/11/2026.	Nomes no vídeo e README; checklist de submissão.
Entregáveis	Código(s) do servidor e vídeo-relatório de até 10 minutos; enviar somente o link do vídeo no Classroom.	Repositório/pasta com código; link testado em janela anônima.
Sockets em C	Aplicação cliente-servidor de arquivos usando API de sockets na linguagem C.	Código compilável e comandos de build.
Concorrência	Múltiplos clientes conectados e atendidos simultaneamente.	Thread/processo por conexão; teste com vários clientes.
GET	GET caminho_arquivo baixa do servidor para o cliente; substitui o arquivo existente no cliente; não implementar GET condicional.	Teste de arquivo novo e teste de sobrescrita.
PUT	PUT caminho_arquivo envia arquivo local para o servidor; substitui arquivo existente no servidor.	Teste de arquivo novo e teste de sobrescrita.
Erros	Servidor responde conforme o método e envia o código do erro ao cliente.	Tabela de códigos e testes de arquivo inexistente, permissão e formato inválido.
CORE/topologia	Redes de acesso e núcleo; servidor em uma rede de acesso e cliente em outra.	Arquivo da sessão/topologia e captura da tela.
Gargalo	Escolher um enlace entre cliente e servidor e configurar vazão e latência, sem configurar erro.	Parâmetros no relatório e no arquivo de configuração.
Validação de rede	ping e traceroute para RTT fim a fim/por salto; iperf entre cliente e servidor para vazão.	Saídas salvas em arquivos.
Planejamento	Variar tamanhos de arquivos, operações, clientes concorrentes e vazão/latência de duas tecnologias comuns de acesso à Internet.	Matriz de cenários antes da execução.
Repetição	Repetir cada configuração n vezes; em ambiente controlado, usar n > 10.	CSV com pelo menos 11 observações por configuração.
Telemetria	Para GET e PUT: tempo de resposta e	CSV e script de coleta.

	vazão obtida.	
Análise	Média, mediana, desvio padrão, intervalo de confiança; gráficos relevantes; gargalos e pontos de saturação.	Tabela estatística, barras/linhas/CDF/boxplot e discussão.
Vídeo	Introdução/motivação, o que-como-porque-onde, resultados, conclusões; incluir contribuição de cada integrante e autoavaliação.	Vídeo ≤ 10 min hospedado na nuvem; enviar apenas link.

2. Arquitetura e melhor abordagem de solução

A abordagem recomendada separa o problema em quatro camadas: protocolo de aplicação, transporte TCP, execução concorrente e experimento. Isso evita misturar falhas de implementação com efeitos de rede e torna possível explicar os resultados.

2.1 Protocolo de aplicação recomendado

Use TCP e defina um protocolo simples, documentado e orientado a mensagens. O cliente abre uma conexão para cada requisição, envia uma linha de comando e o servidor responde com um cabeçalho de status. Para não depender de delimitadores dentro do arquivo, envie o tamanho do conteúdo antes dos bytes do arquivo.

```
Requisição:  <MÉTODO> <caminho_relativo> <tamanho_em_bytes>\n
Resposta:    <código> <mensagem> <tamanho_em_bytes>\n
Conteúdo:    exatamente N bytes, quando aplicável
```

Exemplo: PUT dados/video.bin 10485760 seguido de 10.485.760 bytes. O servidor responde 200 OK após gravar e fechar o arquivo. Em GET, o cliente envia tamanho 0 e recebe o tamanho real no cabeçalho de resposta. O caminho deve ser relativo a uma raiz de dados configurada, nunca um caminho absoluto recebido diretamente do usuário.

Código	Uso sugerido	Quando usar
200	OK	GET/PUT concluído.
400	BAD_REQUEST	Linha inválida, método desconhecido ou tamanho inválido.
404	NOT_FOUND	GET de arquivo inexistente.
409	CONFLICT	Conflito de nome/política, se adotado.
413	PAYLOAD_TOO_LARGE	Limite de arquivo excedido.
500	INTERNAL_ERROR	Falha de I/O ou erro interno.
503	BUSY/UNAVAILABLE	Limite de conexões/recursos, se adotado.

2.2 Estrutura de código

```
trabalho1/
  src/server.c        # socket, bind, listen, accept, concorrência
  src/client.c        # conexão, envio de comando, GET/PUT
  src/protocol.c      # send_all/recv_all, cabeçalhos, validação
```

```
src/protocol.h
src/Makefile
scripts/run_client.sh
experiments/          # CSV bruto e resultados processados
analysis/analyze.py   # estatística e gráficos
README.md
```

No servidor, use socket(AF_INET, SOCK_STREAM, 0), setsockopt(SO_REUSEADDR), bind, listen e um loop accept. Para cada conexão, crie uma thread POSIX no Linux/WSL com pthread_create e pthread_detach, ou um processo separado. Como o CORE executará nós Linux, pthreads é a escolha mais direta. Garanta send_all/recv_all: send e recv podem transferir menos bytes do que o solicitado.

Proteja os dados compartilhados. A alternativa mais simples e reprodutível é dar a cada requisição um arquivo temporário exclusivo e fazer rename atômico ao concluir, evitando que um GET veja um arquivo parcialmente escrito. Se múltiplos PUTs do mesmo nome forem permitidos, declare a política: “o último PUT concluído vence” e registre essa limitação.

Imponha uma raiz de armazenamento, normalize o caminho e rejeite “..”, caminhos absolutos e separadores inesperados. Isso é necessário para evitar path traversal e também facilita reproduzir os testes dentro dos nós do CORE.

3. Preparação do Windows: rota principal WSL2 + Docker Desktop

A instalação deve ser feita em PowerShell como administrador quando indicado e, depois, no terminal Ubuntu/WSL. A Microsoft documenta wsl --install para Windows 10 versão 2004 (build 19041+) e Windows 11; distribuições novas usam WSL2 por padrão. [1]

3.1 Pré-requisitos do computador

- Windows 11 ou Windows 10 2004/build 19041 ou superior.
- Virtualização habilitada no BIOS/UEFI e espaço livre suficiente para WSL, imagens Docker e arquivos de teste. Reserve pelo menos 8 GB de RAM e 30 GB livres; mais memória melhora a execução de muitos clientes.
- Permissão de administrador para habilitar recursos do Windows e instalar Docker Desktop.
- Windows Terminal e VS Code são recomendados, mas o editor pode ser substituído por qualquer editor.

3.2 Instalar WSL2 e Ubuntu

```
# PowerShell (Administrador)
wsl --install
# Reinicie o Windows quando solicitado. Na primeira abertura do Ubuntu, crie usuário e
senha.

wsl --update
wsl --set-default-version 2
wsl --list --verbose
```

Se wsl --install mostrar apenas a ajuda, liste as distribuições e instale explicitamente:

```
wsl --list --online
wsl --install -d Ubuntu-22.04
wsl --set-version Ubuntu-22.04 2
wsl --set-default Ubuntu-22.04
```

Critério de aceite: wsl --list --verbose deve mostrar a distribuição Ubuntu com VERSION 2. Guarde o nome exato da distribuição para os comandos seguintes. Não misture comandos de instalação Linux diretamente no PowerShell.

3.3 Instalar ferramentas Linux dentro do Ubuntu

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y build-essential gcc make git python3 python3-pip python3-venv \
  iproute2 iputils-ping traceroute iperf3 net-tools procps curl unzip

gcc --version
git --version
python3 --version
iperf3 --version
```

O código-fonte deve ficar no sistema de arquivos Linux, por exemplo ~/trabalho1-redes, e não em /mnt/c, para reduzir problemas de desempenho e permissões durante builds e muitos acessos a arquivos.

3.4 Instalar Docker Desktop e integrar com WSL2

Baixe o instalador somente do site oficial do Docker Desktop para Windows: https://www.docker.com/products/docker-desktop/. Durante a instalação, selecione o backend WSL2 quando oferecido. Abra Docker Desktop e confirme Settings > General > Use the WSL 2 based engine. Depois, em Settings > Resources > WSL Integration, habilite a integração com Ubuntu. A documentação oficial alerta para não manter outro Docker Engine instalado dentro da distribuição WSL, pois isso pode causar conflitos. [3]

```
# PowerShell ou Ubuntu, após abrir o Docker Desktop
docker version
docker run --rm hello-world
```

Critério de aceite: docker version apresenta Client e Server, e hello-world termina com sucesso. Se o comando funcionar no PowerShell mas não no Ubuntu, habilite a distribuição em Resources > WSL Integration e reinicie o Docker Desktop.

4. Instalação do CORE no ambiente Windows

O CORE requer um ambiente Linux com Python 3.10+, pip, venv, suporte Tcl/Tk, iproute2 e nftables. O método com contêiner é o mais próximo do Anexo I e evita instalar todas as dependências do CORE diretamente no Ubuntu. [2]

4.1 Rota recomendada: CORE no contêiner Docker

```
mkdir -p ~/trabalho1-redes/tools
cd ~/trabalho1-redes/tools
git clone https://github.com/coreemu/core.git
cd core
```

```
# Construir as imagens, conforme o Anexo I e a documentação atual
docker build -t emane-python -f dockerfiles/Dockerfile.emane-python .
docker build -t core -f dockerfiles/Dockerfile.ubuntu .
```

O primeiro build pode demorar e consumir bastante disco. Se a versão do repositório não tiver Dockerfile.ubuntu, liste dockerfiles e use o Dockerfile suportado pela revisão clonada, sem inventar o nome do arquivo.

```
# Em uma sessão WSL com suporte gráfico WSLg, iniciar o daemon
docker run -itd --name core -e DISPLAY \
  -v /tmp/.X11-unix:/tmp/.X11-unix:rw \
  --privileged --init \
  --entrypoint /opt/core/venv/bin/core-daemon core

# Abrir a GUI no contêiner
docker exec -it core core-gui
```

No Windows 11, o WSLg normalmente fornece a exibição gráfica. Se a GUI não abrir, confirme echo $DISPLAY, teste o contêiner com docker logs core e, se necessário, use a instalação nativa do CORE dentro do Ubuntu WSL ou a rota Vagrant da Seção 4.2. Não desative a segurança do host permanentemente.

Se o contêiner já existir: docker start core e docker exec -it core core-gui. Para reconstruir do zero: docker rm -f core, mantendo as imagens se não for necessário refazer o build.

4.2 Rota alternativa do Anexo I: Vagrant + VirtualBox

Use esta rota apenas se a rota WSL2/Docker não funcionar ou se o laboratório exigir Vagrant. Instale VirtualBox antes do Vagrant. O provedor VirtualBox precisa que o VirtualBox esteja instalado separadamente; a documentação do Vagrant lista compatibilidade com versões 7.0–7.2, entre outras. [4]

Downloads oficiais: VirtualBox: https://www.virtualbox.org/wiki/Downloads ; Vagrant: https://developer.hashicorp.com/vagrant/downloads. Após instalar, abra um novo PowerShell e valide:

```
vboxmanage --version
vagrant --version
mkdir C:\Users\<usuario>\trabalho1-vagrant
cd C:\Users\<usuario>\trabalho1-vagrant
vagrant init hashicorp/bionic64
vagrant up
vagrant reload
vagrant ssh
```

Dentro da VM, reproduza o Anexo I. O comando hashicorp/bionic64 é legado; se a imagem não baixar ou apresentar incompatibilidade, substitua por uma box Ubuntu suportada e ajuste os comandos. O CORE atual documenta Ubuntu 22.04 no Dockerfile, portanto não trate Bionic como requisito funcional do trabalho.

```
sudo apt update
sudo apt install -y apt-transport-https ca-certificates curl software-properties-common git
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu bionic
stable"
```

```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io
git clone https://github.com/coreemu/core.git && cd core
sudo docker build -t emane-python -f dockerfiles/Dockerfile.emane-python .
sudo docker build -t core -f dockerfiles/Dockerfile.ubuntu .
```

Atenção: os comandos acima reproduzem o anexo, mas podem exigir ajuste de repositório e chave em uma distribuição antiga. Prefira a documentação atual do CORE e Docker quando houver conflito. [2]

5. Implementar, compilar e testar a aplicação em C

5.1 Compilação

```
cd ~/trabalho1-redes
# exemplo de Makefile
gcc -Wall -Wextra -O2 -pthread -o server src/server.c src/protocol.c
gcc -Wall -Wextra -O2 -o client src/client.c src/protocol.c
./server --bind 0.0.0.0 --port 9000 --root ./server-data
```

Escolha uma porta não privilegiada, como 9000. O servidor deve imprimir porta, raiz de dados, número de conexões aceitas e erros. O cliente deve imprimir método, arquivo, bytes, tempo de resposta e vazão em MiB/s.

5.2 Testes funcionais mínimos

Teste	Procedimento	Resultado esperado
GET sucesso	Coloque arquivo no servidor; execute client GET.	Arquivo aparece no cliente com mesmo tamanho e hash.
GET sobrescrita	Altere o arquivo local e repita GET.	Conteúdo local é substituído pelo conteúdo do servidor.
PUT sucesso	Crie arquivo local; execute client PUT.	Arquivo aparece no servidor com mesmo tamanho e hash.
PUT sobrescrita	Altere o arquivo local e repita PUT.	Conteúdo no servidor é substituído.
Arquivo ausente	GET de nome inexistente.	Cliente recebe 404; processo não cai.
Comando inválido	Envie método desconhecido ou cabeçalho malformado.	Cliente recebe 400; conexão é encerrada com segurança.
Concorrência	Lance 2, 8 e 32 clientes em paralelo.	Todos concluem; servidor continua aceitando conexões.

Compare arquivos com SHA-256, não apenas com o nome:

```
sha256sum server-data/arquivo.bin
sha256sum client-data/arquivo.bin
# concorrência simples
seq 1 8 | xargs -P 8 -I{} ./client 10.0.0.2 9000 PUT client-data/arq{}.bin
```

6. Montar a topologia no CORE

Crie uma topologia com duas redes de acesso e uma rede de núcleo entre elas. Um desenho mínimo é: cliente — roteador de acesso A — roteador de núcleo — roteador de acesso B — servidor. Coloque

cliente e servidor em sub-redes diferentes. Se houver apenas um roteador no núcleo, ele ainda deve ser identificado como núcleo no relatório.

Elemento	Exemplo	Observação
Acesso do cliente	10.0.1.0/24	Nó cliente C1.
Núcleo	10.0.2.0/24	Enlace onde será aplicado o gargalo.
Acesso do servidor	10.0.3.0/24	Nó servidor S1.
Gargalo	R1–R2	Configure vazão e latência; não configure perda/erro.
Serviços	server em S1; gerador em C1	iperf3 server pode ficar em S1 durante calibração.

6.1 Configurar nós

No nó servidor, copie o binário server e a pasta server-data; no cliente, copie client e os arquivos de teste. Defina endereços IP pelo mecanismo de configuração da sessão ou do CORE e anote a saída de ip addr em cada nó. Inicie o servidor antes dos testes de aplicação.

```
# em cada nó CORE
ip addr
ip route
ping -c 10 <ip_do_outro_nó>
traceroute -n <ip_do_outro_nó>
```

6.2 Configurar o gargalo

Escolha dois perfis de acesso representativos, por exemplo: Perfil A “banda larga fixa” com 100 Mbit/s e 10 ms de atraso em cada sentido; Perfil B “acesso móvel” com 20 Mbit/s e 40 ms de atraso em cada sentido. Os valores são parâmetros experimentais arbitrários, não uma afirmação de que toda tecnologia tenha esses números. Use os mesmos valores em cada repetição de um cenário.

No CORE, configure a largura de banda e o atraso no enlace selecionado. Não configure erro ou perda, pois o enunciado proíbe. Registre se o atraso é unidirecional ou aplicado nos dois sentidos; essa distinção é essencial para interpretar o RTT.

```
# validação no enlace
# servidor iperf3
iperf3 -s
# cliente iperf3
iperf3 -c <ip_servidor> -t 20 -i 1

# guardar evidências
ping -c 30 <ip_servidor> | tee ping-perfilA.txt
traceroute -n <ip_servidor> | tee traceroute-perfilA.txt
iperf3 -c <ip_servidor> -t 20 -J > iperf-perfilA.json
```

Critério de aceite: o RTT observado deve ser compatível com a soma dos atrasos configurados e dos atrasos de processamento dos saltos; a vazão do iperf3 deve ficar próxima do limite configurado, descontando overhead de TCP e efeitos do RTT. Se não ficar, verifique se o gargalo foi aplicado no enlace correto, se o tráfego atravessa o enlace e se o Docker/CORE não bloqueou forwarding.

7. Planejamento experimental

Faça o planejamento antes de executar e congele a matriz. O enunciado pede tamanhos distintos, operações, concorrência e parâmetros de rede. A matriz abaixo é uma proposta equilibrada, suficientemente grande para revelar tendências sem gerar uma campanha impossível de concluir.

7.1 Fatores e níveis sugeridos

Fator	Níveis propostos	Justificativa
Aplicação	GET e PUT	São os dois métodos exigidos; medir ambos separa download e upload.
Tamanho pequeno	1 MiB	Representa documento/imagem pequena e reduz tempo de depuração.
Tamanho grande	100 MiB	Representa mídia/arquivo grande e evidencia o gargalo.
Concorrência baixa	1 cliente	Linha de base.
Concorrência média	8 clientes	Carga concorrente controlada.
Concorrência alta	32 clientes	Revela fila, contenção e saturação; reduza se o hardware não suportar.
Rede A	100 Mbit/s, 10 ms	Perfil de acesso fixo hipotético.
Rede B	20 Mbit/s, 40 ms	Perfil de acesso móvel/limitado hipotético.
Repetições	11 ou 15 por célula	Atende n > 10; use 15 se o tempo permitir.

Com 2 operações × 2 tamanhos × 3 concorrências × 2 redes × 11 repetições, a campanha tem 264 observações de célula/execução. Se cada execução paralela gerar uma linha por cliente, registre também o identificador da requisição e agregue depois por célula.

7.2 Arquivos típicos de duas aplicações

Documente a associação, sem afirmar que o tamanho é universal: (a) aplicação de armazenamento/sincronização em nuvem: arquivo de documento ou pacote de projeto de 1 MiB; (b) aplicação de streaming/mídia ou compartilhamento de arquivos: bloco de mídia de 100 MiB. O objetivo acadêmico é criar tamanhos pequenos e grandes que exponham comportamentos diferentes.

7.3 Formato do CSV

```
run_id,perfil,bandwidth_mbps,latency_ms,operation,file_size_bytes,concurrency,client_id,resp
onse_ms,throughput_mbps,status,error
001,A,100,10,GET,1048576,1,1,12.4,677.4,200,
002,A,100,10,PUT,104857600,8,3,8420.1,99.7,200,
```

Use tempo monotônico no cliente (clock_gettime(CLOCK_MONOTONIC, ...)). Calcule throughput = (bytes × 8) / (tempo_em_segundos × 1.000.000), em Mbit/s. Para GET/PUT, defina claramente se o tempo inclui conexão TCP, handshake e escrita/leitura do arquivo; a recomendação é incluir a operação completa observada pelo cliente e informar isso no vídeo.

8. Execução automatizada e coleta

8.1 Sequência por configuração

1. Reinicie ou limpe o estado dos diretórios para impedir que cache ou arquivos antigos contaminem a medição.
2. Configure o perfil de rede e confirme com ping, traceroute e iperf3. Salve as saídas.
3. Inicie o servidor e confirme que a porta está ouvindo com ss -lntp.
4. Para cada operação e tamanho, lance o número de clientes definido. Use arquivos previamente gerados e hash registrados.
5. Colete uma linha por requisição e uma linha de resumo por célula. Marque falhas sem apagá-las; status e erro fazem parte do resultado.
6. Repita pelo menos 11 vezes. Randomize a ordem das células ou alterne perfis para reduzir efeito de aquecimento, mas mantenha o mesmo protocolo.

```
# gerar arquivos controlados
fallocate -l 1M client-data/file_1MiB.bin
fallocate -l 100M client-data/file_100MiB.bin
sha256sum client-data/* > checksums.txt

# exemplo de repetição no nó cliente
for run in $(seq 1 11); do
  ./client --csv results.csv --id "$run" <args-do-cenario>
done
```

9. Processamento e análise

Use Python com pandas, numpy e matplotlib/seaborn. Instale somente no ambiente do projeto:

```
python3 -m venv .venv
source .venv/bin/activate
pip install pandas numpy scipy matplotlib seaborn
python analysis/analyze.py experiments/results.csv
```

Para cada célula, calcule n válido, média, mediana, desvio padrão amostral, mínimo, máximo e intervalo de confiança de 95%. Para n pequeno ou distribuição assimétrica, informe também mediana e intervalo percentílico; não esconda outliers.

```
# IC 95% para a média, com t de Student
media ± t_(0.975, n-1) * desvio_padrao / sqrt(n)
```

9.1 Gráficos mínimos

- Barras ou pontos com IC: tempo médio por operação e tamanho, separado por perfil de rede.
- Linhas: throughput versus número de clientes, com uma linha para cada tamanho.
- Boxplot: distribuição do tempo de resposta entre as 11+ repetições.
- CDF: tempo de resposta, especialmente para baixa, média e alta concorrência.
- Heatmap opcional: tempo médio para todas as combinações de concorrência e parâmetros.

9.2 Como identificar gargalos e saturação

Compare o throughput da aplicação com o iperf3. Se o iperf3 se aproxima do limite, mas GET/PUT fica abaixo com um cliente, investigue custo de disco, protocolo, buffers ou implementação. Se um cliente é próximo do limite e 32 clientes não aumentam throughput, mas aumentam o tempo de resposta, há saturação do enlace, do servidor, do disco ou da fila. Se o throughput cai somente em GET ou somente em PUT, compare direção, leitura/escrita e política de sobrescrita.

Não conclua “a rede é o gargalo” apenas pela queda de desempenho: sustente a conclusão com vazão do iperf3, RTT, CPU/memória do nó, tamanho do arquivo e número de clientes. Registre limitações: emulação sem perda, hardware do hospedeiro e overhead do Docker/WSL2.

10. Vídeo-relatório de até 10 minutos

Tempo	Conteúdo	Evidência na tela
0:00–1:00	Motivação e problema de cliente-servidor de arquivos.	Objetivo e requisitos.
1:00–2:30	O que foi desenvolvido, por quê e onde.	Arquitetura, protocolo e topologia CORE.
2:30–4:00	Implementação.	Código, GET/PUT, concorrência e códigos de erro.
4:00–6:30	Experimento.	Matriz, gargalo, ping/traceroute/iperf3 e execução.
6:30–8:30	Resultados.	Gráficos, estatísticas, gargalos e saturação.
8:30–9:20	Contribuições individuais.	O que cada integrante fez.
9:20–10:00	Autoavaliação e conclusões.	Limitações, aprendizados e conclusão.

Hospede o vídeo em uma nuvem com permissão de visualização para quem possui o link. Teste o link em uma janela anônima. No Classroom, envie somente o link, conforme o enunciado; não envie o arquivo de vídeo.

11. Checklist final de entrega

- Grupo com no máximo três alunos e nomes/contribuições definidos.
- Código do servidor e cliente em C, com README e instruções de compilação.
- GET baixa e sobrescreve arquivo no cliente; PUT envia e sobrescreve arquivo no servidor.
- Múltiplos clientes atendidos simultaneamente e teste de concorrência registrado.
- Códigos de erro enviados pelo servidor e testados.
- Topologia CORE com duas redes de acesso, núcleo, cliente e servidor em acessos distintos.
- Um enlace configurado com vazão e latência, sem erro/perda.
- Ping, traceroute e iperf3 salvos para validar a emulação.
- Matriz com 2 tamanhos, GET/PUT, baixa/média/alta concorrência, 2 perfis de rede e n > 10.
- CSV bruto preservado; tempo de resposta e vazão coletados por operação.
- Estatísticas: média, mediana, desvio padrão e IC; gráficos e discussão de gargalos/saturação.

- Vídeo com introdução, desenvolvimento, resultados, conclusões, contribuições e autoavaliação; duração ≤ 10 min.
- Link do vídeo testado e enviado sozinho no Classroom até 05/11/2026.

12. Solução de problemas

Sintoma	Diagnóstico e ação
docker: command not found no Ubuntu	Abra Docker Desktop, habilite WSL Integration para Ubuntu e abra novo terminal. Não instale um segundo Docker Engine dentro do WSL.
CORE GUI não abre	Confirme WSLg/echo $DISPLAY; consulte docker logs core. Como alternativa, use CORE nativo no Ubuntu WSL ou Vagrant.
iperf3 abaixo da vazão	Confirme enlace do gargalo, rota, unidade Mbit/s versus MiB/s, duração do teste e forwarding.
traceroute não mostra saltos	Instale traceroute no nó e use traceroute -n; confirme que o tráfego realmente atravessa roteadores do núcleo.
cliente trava em recv	Implemente recv_all, limite de cabeçalho, timeout e fechamento explícito. Verifique se o servidor enviou N bytes.
arquivos diferentes	Compare tamanho e sha256sum; confira sobrescrita atômica e não misture diretórios de cliente/servidor.
muitos clientes falham	Verifique backlog, limite de descritores, pthread_detach, memória e se o cenário excede a capacidade do hospedeiro.
Vagrant Bionic falha	Trate como incompatibilidade de box legada; use a rota WSL2/Docker ou uma box Ubuntu suportada, documentando a mudança.

Apêndice A – Código completo de referência

Os arquivos abaixo formam uma implementação mínima, completa e compilável para o trabalho. O grupo deve entendê-los, testar, adaptar e documentar as próprias decisões. O código usa TCP, uma thread por conexão, caminhos relativos à raiz do servidor, transferência por tamanho explícito e medição no cliente. Em Linux/WSL/CORE, compile com o Makefile apresentado.

A.1 src/protocol.h

```
#ifndef PROTOCOL_H
#define PROTOCOL_H

#include <stddef.h>
#include <stdint.h>
#include <sys/types.h>

#define PROTOCOL_MAX_LINE 4096
#define PROTOCOL_MAX_PATH 1024
#define PROTOCOL_MAX_FILE (1024ULL * 1024ULL * 1024ULL)

/* Envia exatamente len bytes, salvo erro. */
int send_all(int fd, const void *buf, size_t len);
```

```
/* Recebe exatamente len bytes, salvo EOF ou erro. */
int recv_all(int fd, void *buf, size_t len);

/* Lê uma linha terminada por LF e adiciona NUL. */
ssize_t recv_line(int fd, char *buf, size_t cap);

/* Remove CR/LF final e espaços laterais simples. */
void trim_line(char *s);

/* Retorna 0 se o caminho for relativo e seguro. */
int validate_relative_path(const char *path);

#endif
```

A.2 src/protocol.c

```
#define _POSIX_C_SOURCE 200809L
#include "protocol.h"

#include <ctype.h>
#include <errno.h>
#include <string.h>
#include <sys/socket.h>

int send_all(int fd, const void *buf, size_t len) {
    const char *p = (const char *)buf;
    while (len > 0) {
        ssize_t n = send(fd, p, len, MSG_NOSIGNAL);
        if (n < 0) {
            if (errno == EINTR) continue;
            return -1;
        }
        if (n == 0) return -1;
        p += n;
        len -= (size_t)n;
    }
    return 0;
}

int recv_all(int fd, void *buf, size_t len) {
    char *p = (char *)buf;
    while (len > 0) {
        ssize_t n = recv(fd, p, len, 0);
        if (n < 0) {
            if (errno == EINTR) continue;
            return -1;
        }
        if (n == 0) return 1; /* conexão fechada antes do esperado */
        p += n;
        len -= (size_t)n;
    }
    return 0;
}

ssize_t recv_line(int fd, char *buf, size_t cap) {
    if (cap == 0) return -1;
    size_t used = 0;
    while (used + 1 < cap) {
```

```
        char c;
        ssize_t n = recv(fd, &c, 1, 0);
        if (n < 0) {
            if (errno == EINTR) continue;
            return -1;
        }
        if (n == 0) break;
        buf[used++] = c;
        if (c == '\n') break;
    }
    buf[used] = '\0';
    if (used == 0) return 0;
    if (used + 1 >= cap && buf[used - 1] != '\n') return -2;
    return (ssize_t)used;
}

void trim_line(char *s) {
    size_t n = strlen(s);
    while (n > 0 && (s[n - 1] == '\n' || s[n - 1] == '\r' || isspace((unsigned char)s[n -
1]))) {
        s[--n] = '\0';
    }
    size_t start = 0;
    while (s[start] && isspace((unsigned char)s[start])) start++;
    if (start > 0) memmove(s, s + start, strlen(s + start) + 1);
}

int validate_relative_path(const char *path) {
    if (!path || !*path) return 0;
    if (path[0] == '/' || path[0] == '\\') return 0;
    if (strstr(path, "..") != NULL) return 0;
    if (strchr(path, '\\') != NULL) return 0;
    return strlen(path) < PROTOCOL_MAX_PATH;
}
```

A.3 src/server.c

```
#define _POSIX_C_SOURCE 200809L
#include "protocol.h"

#include <arpa/inet.h>
#include <errno.h>
#include <fcntl.h>
#include <limits.h>
#include <netinet/in.h>
#include <pthread.h>
#include <signal.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

#define DEFAULT_PORT 9000
#define DEFAULT_ROOT "./server-data"
#define IO_BUFFER 65536
```

```
typedef struct {
    int fd;
    char root[PATH_MAX];
} client_arg_t;

static int send_status(int fd, int code, const char *message, uint64_t size) {
    char line[256];
    int n = snprintf(line, sizeof(line), "%d %s %llu\n", code, message,
                     (unsigned long long)size);
    if (n < 0 || (size_t)n >= sizeof(line)) return -1;
    return send_all(fd, line, (size_t)n);
}

static int join_safe_path(const char *root, const char *relative, char *out, size_t cap) {
    if (!validate_relative_path(relative)) return -1;
    int n = snprintf(out, cap, "%s/%s", root, relative);
    return (n < 0 || (size_t)n >= cap) ? -1 : 0;
}

static int handle_get(int fd, const char *root, const char *path) {
    char full[PATH_MAX];
    if (join_safe_path(root, path, full, sizeof(full)) < 0)
        return send_status(fd, 400, "BAD_REQUEST", 0);

    int file = open(full, O_RDONLY);
    if (file < 0) {
        return send_status(fd, errno == ENOENT ? 404 : 500,
                           errno == ENOENT ? "NOT_FOUND" : "OPEN_ERROR", 0);
    }
    struct stat st;
    if (fstat(file, &st) < 0 || !S_ISREG(st.st_mode) || st.st_size < 0) {
        close(file);
        return send_status(fd, 500, "STAT_ERROR", 0);
    }
    uint64_t size = (uint64_t)st.st_size;
    if (size > PROTOCOL_MAX_FILE) {
        close(file);
        return send_status(fd, 413, "PAYLOAD_TOO_LARGE", 0);
    }
    if (send_status(fd, 200, "OK", size) < 0) { close(file); return -1; }

    char buf[IO_BUFFER];
    uint64_t sent = 0;
    while (sent < size) {
        ssize_t n = read(file, buf, sizeof(buf));
        if (n < 0) { if (errno == EINTR) continue; close(file); return -1; }
        if (n == 0) break;
        if (send_all(fd, buf, (size_t)n) < 0) { close(file); return -1; }
        sent += (uint64_t)n;
    }
    close(file);
    return sent == size ? 0 : -1;
}

static int make_parent_directory(const char *root) {
    struct stat st;
    if (stat(root, &st) == 0) return S_ISDIR(st.st_mode) ? 0 : -1;
    if (mkdir(root, 0755) < 0 && errno != EEXIST) return -1;
    return 0;
}
```

```
static int handle_put(int fd, const char *root, const char *path, uint64_t size) {
    char full[PATH_MAX], temp[PATH_MAX];
    if (join_safe_path(root, path, full, sizeof(full)) < 0)
        return send_status(fd, 400, "BAD_REQUEST", 0);
    if (size > PROTOCOL_MAX_FILE) return send_status(fd, 413, "PAYLOAD_TOO_LARGE", 0);
    int n = snprintf(temp, sizeof(temp), "%s.tmp.%ld", full, (long)getpid());
    if (n < 0 || (size_t)n >= sizeof(temp)) return send_status(fd, 500, "PATH_ERROR", 0);

    int file = open(temp, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (file < 0) return send_status(fd, 500, "CREATE_ERROR", 0);
    char buf[IO_BUFFER];
    uint64_t received = 0;
    while (received < size) {
        size_t want = (size - received < sizeof(buf)) ? (size_t)(size - received) :
sizeof(buf);
        int rc = recv_all(fd, buf, want);
        if (rc != 0) { close(file); unlink(temp); return -1; }
        size_t written = 0;
        while (written < want) {
            ssize_t w = write(file, buf + written, want - written);
            if (w < 0) { if (errno == EINTR) continue; close(file); unlink(temp); return -1;
}
            written += (size_t)w;
        }
        received += want;
    }
    if (fsync(file) < 0) { close(file); unlink(temp); return send_status(fd, 500,
"SYNC_ERROR", 0); }
    close(file);
    if (rename(temp, full) < 0) { unlink(temp); return send_status(fd, 500, "RENAME_ERROR",
0); }
    return send_status(fd, 200, "OK", size);
}

static void *client_thread(void *opaque) {
    client_arg_t *arg = (client_arg_t *)opaque;
    int fd = arg->fd;
    char root[PATH_MAX];
    snprintf(root, sizeof(root), "%s", arg->root);
    free(arg);

    char line[PROTOCOL_MAX_LINE];
    ssize_t n = recv_line(fd, line, sizeof(line));
    if (n <= 0 || n == -2) { send_status(fd, 400, "BAD_REQUEST", 0); close(fd); return NULL;
}
    trim_line(line);

    char method[16], path[PROTOCOL_MAX_PATH];
    unsigned long long size = 0;
    int fields = sscanf(line, "%15s %1023s %llu", method, path, &size);
    if (fields != 3 || !validate_relative_path(path)) {
        send_status(fd, 400, "BAD_REQUEST", 0); close(fd); return NULL;
    }
    int rc;
    if (strcmp(method, "GET") == 0 && size == 0) rc = handle_get(fd, root, path);
    else if (strcmp(method, "PUT") == 0) rc = handle_put(fd, root, path, (uint64_t)size);
    else rc = send_status(fd, 400, "BAD_REQUEST", 0);
    (void)rc;
    close(fd);
    return NULL;
```

```
}

static void usage(const char *prog) {
    fprintf(stderr, "Uso: %s [--bind IP] [--port PORT] [--root DIRETORIO]\n", prog);
}

int main(int argc, char **argv) {
    signal(SIGPIPE, SIG_IGN);
    const char *bind_ip = "0.0.0.0", *root = DEFAULT_ROOT;
    int port = DEFAULT_PORT;
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "--bind") == 0 && i + 1 < argc) bind_ip = argv[++i];
        else if (strcmp(argv[i], "--port") == 0 && i + 1 < argc) port = atoi(argv[++i]);
        else if (strcmp(argv[i], "--root") == 0 && i + 1 < argc) root = argv[++i];
        else { usage(argv[0]); return 2; }
    }
    if (port < 1 || port > 65535 || make_parent_directory(root) < 0)
{ perror("configuração"); return 1; }

    int server = socket(AF_INET, SOCK_STREAM, 0);
    if (server < 0) { perror("socket"); return 1; }
    int yes = 1; setsockopt(server, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(yes));
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET; addr.sin_port = htons((uint16_t)port);
    if (inet_pton(AF_INET, bind_ip, &addr.sin_addr) != 1) { fprintf(stderr, "IP inválido\
n"); return 1; }
    if (bind(server, (struct sockaddr *)&addr, sizeof(addr)) < 0 || listen(server, 128) < 0)
{ perror("bind/listen"); return 1; }
    printf("server listening on %s:%d root=%s\n", bind_ip, port, root); fflush(stdout);

    for (;;) {
        int fd = accept(server, NULL, NULL);
        if (fd < 0) { if (errno == EINTR) continue; perror("accept"); continue; }
        client_arg_t *arg = calloc(1, sizeof(*arg));
        if (!arg) { close(fd); continue; }
        arg->fd = fd; snprintf(arg->root, sizeof(arg->root), "%s", root);
        pthread_t tid;
        if (pthread_create(&tid, NULL, client_thread, arg) != 0) { close(fd); free(arg);
continue; }
        pthread_detach(tid);
    }
}
```

A.4 src/client.c

```
#define _POSIX_C_SOURCE 200809L
#include "protocol.h"

#include <arpa/inet.h>
#include <errno.h>
#include <fcntl.h>
#include <netdb.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/stat.h>
#include <time.h>
```

```
#include <unistd.h>

#define IO_BUFFER 65536

static double now_seconds(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec + (double)ts.tv_nsec / 1e9;
}

static int connect_to(const char *host, const char *port) {
    struct addrinfo hints = {0}, *list = NULL;
    hints.ai_family = AF_INET; hints.ai_socktype = SOCK_STREAM;
    if (getaddrinfo(host, port, &hints, &list) != 0) return -1;
    int fd = -1;
    for (struct addrinfo *p = list; p; p = p->ai_next) {
        fd = socket(p->ai_family, p->ai_socktype, p->ai_protocol);
        if (fd >= 0 && connect(fd, p->ai_addr, p->ai_addrlen) == 0) break;
        if (fd >= 0) { close(fd); fd = -1; }
    }
    freeaddrinfo(list); return fd;
}

static int parse_response(const char *line, int *code, uint64_t *size) {
    char msg[64]; unsigned long long n;
    if (sscanf(line, "%d %63s %llu", code, msg, &n) != 3) return -1;
    *size = (uint64_t)n; return 0;
}

static int do_get(int fd, const char *remote_path, const char *local_path, uint64_t *bytes)
{
    char request[PROTOCOL_MAX_LINE];
    int n = snprintf(request, sizeof(request), "GET %s 0\n", remote_path);
    if (n < 0 || (size_t)n >= sizeof(request) || send_all(fd, request, (size_t)n) < 0)
return -1;
    char line[PROTOCOL_MAX_LINE]; if (recv_line(fd, line, sizeof(line)) <= 0) return -1;
    trim_line(line); int code; uint64_t size;
    if (parse_response(line, &code, &size) < 0) return -1;
    if (code != 200) { fprintf(stderr, "server: %s\n", line); return code; }
    char temp[PROTOCOL_MAX_PATH + 32]; snprintf(temp, sizeof(temp), "%s.tmp", local_path);
    int out = open(temp, O_WRONLY | O_CREAT | O_TRUNC, 0644); if (out < 0) return -1;
    char buf[IO_BUFFER]; uint64_t got = 0;
    while (got < size) {
        size_t want = (size - got < sizeof(buf)) ? (size_t)(size - got) : sizeof(buf);
        int rc = recv_all(fd, buf, want); if (rc != 0) { close(out); unlink(temp); return -
1; }
        size_t off = 0; while (off < want) { ssize_t w = write(out, buf + off, want - off);
if (w < 0) { if (errno == EINTR) continue; close(out); unlink(temp); return -1; } off +=
(size_t)w; }
        got += want;
    }
    close(out); if (rename(temp, local_path) < 0) { unlink(temp); return -1; }
    *bytes = got; return 0;
}

static int do_put(int fd, const char *remote_path, const char *local_path, uint64_t *bytes)
{
    int in = open(local_path, O_RDONLY); if (in < 0) return -1;
    struct stat st; if (fstat(in, &st) < 0 || st.st_size < 0) { close(in); return -1; }
    uint64_t size = (uint64_t)st.st_size;
    char request[PROTOCOL_MAX_LINE]; int n = snprintf(request, sizeof(request), "PUT %s
```

```
%llu\n", remote_path, (unsigned long long)size);
    if (n < 0 || (size_t)n >= sizeof(request) || send_all(fd, request, (size_t)n) < 0)
{ close(in); return -1; }
    char buf[IO_BUFFER]; uint64_t sent = 0;
    while (sent < size) { ssize_t r = read(in, buf, sizeof(buf)); if (r < 0) { if (errno ==
EINTR) continue; close(in); return -1; } if (r == 0) break; if (send_all(fd, buf, (size_t)r)
< 0) { close(in); return -1; } sent += (uint64_t)r; }
    close(in); if (sent != size) return -1;
    char line[PROTOCOL_MAX_LINE]; if (recv_line(fd, line, sizeof(line)) <= 0) return -1;
trim_line(line);
    int code; uint64_t acknowledged; if (parse_response(line, &code, &acknowledged) < 0)
return -1;
    if (code != 200) fprintf(stderr, "server: %s\n", line);
    *bytes = sent; return code == 200 ? 0 : code;
}

static void usage(const char *p) { fprintf(stderr, "Uso: %s HOST PORT GET|PUT REMOTE_PATH
[LOCAL_PATH]\n", p); }

int main(int argc, char **argv) {
    if (argc < 5 || (strcmp(argv[3], "GET") != 0 && strcmp(argv[3], "PUT") != 0))
{ usage(argv[0]); return 2; }
    const char *host = argv[1], *port = argv[2], *method = argv[3], *remote = argv[4];
    const char *local = argc >= 6 ? argv[5] : remote;
    if (!validate_relative_path(remote) || !local || !*local) { fprintf(stderr, "caminho
inválido\n"); return 2; }
    int fd = connect_to(host, port); if (fd < 0) { perror("connect"); return 1; }
    uint64_t bytes = 0; double start = now_seconds();
    int rc = strcmp(method, "GET") == 0 ? do_get(fd, remote, local, &bytes) : do_put(fd,
remote, local, &bytes);
    double elapsed = now_seconds() - start; close(fd);
    if (rc != 0) return rc > 0 ? 1 : 1;
    double mbps = elapsed > 0 ? ((double)bytes * 8.0 / elapsed / 1000000.0) : 0.0;
    printf("operation=%s file=%s bytes=%llu response_ms=%.3f throughput_mbps=%.3f
status=200\n", method, remote, (unsigned long long)bytes, elapsed * 1000.0, mbps);
    return 0;
}
```

A.5 Makefile

```
CC ?= gcc
CFLAGS ?= -Wall -Wextra -O2 -std=c11
LDFLAGS ?=

all: server client

server: src/server.o src/protocol.o
    $(CC) $(CFLAGS) -o $@ $^ -pthread $(LDFLAGS)

client: src/client.o src/protocol.o
    $(CC) $(CFLAGS) -o $@ $^ $(LDFLAGS)

src/%.o: src/%.c src/protocol.h
    $(CC) $(CFLAGS) -Isrc -c -o $@ $<

clean:
    rm -f server client src/*.o
```

```
.PHONY: all clean
```

A.6 Script de análise: analysis/analyze.py

```
#!/usr/bin/env python3
import argparse
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats


def ci95(values):
    values = np.asarray(values, dtype=float)
    n = len(values)
    if n < 2:
        return np.nan
    return stats.t.ppf(0.975, n - 1) * values.std(ddof=1) / np.sqrt(n)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('csv')
    ap.add_argument('--out', default='analysis-output')
    args = ap.parse_args()
    out = Path(args.out); out.mkdir(parents=True, exist_ok=True)
    df = pd.read_csv(args.csv)
    required = {'operation', 'response_ms', 'throughput_mbps'}
    missing = required - set(df.columns)
    if missing: raise SystemExit(f'colunas ausentes: {sorted(missing)}')
    df = df[df.get('status', 200).astype(str) == '200'].copy()
    group_cols = [c for c in ['perfil', 'bandwidth_mbps', 'latency_ms', 'operation',
'file_size_bytes', 'concurrency'] if c in df.columns]
    summary = df.groupby(group_cols).agg(
        n=('response_ms', 'count'),
        response_mean_ms=('response_ms', 'mean'),
        response_median_ms=('response_ms', 'median'),
        response_std_ms=('response_ms', 'std'),
        response_min_ms=('response_ms', 'min'),
        response_max_ms=('response_ms', 'max'),
        throughput_mean_mbps=('throughput_mbps', 'mean'),
        throughput_median_mbps=('throughput_mbps', 'median'),
        throughput_std_mbps=('throughput_mbps', 'std'),
    ).reset_index()
    summary['response_ci95_ms'] = df.groupby(group_cols)
['response_ms'].apply(ci95).to_numpy()
    summary.to_csv(out / 'summary.csv', index=False)
    sns.set_theme(style='whitegrid')
    if 'concurrency' in df.columns:
        plt.figure(figsize=(9, 5))
        sns.lineplot(data=df, x='concurrency', y='throughput_mbps', hue='operation',
style='file_size_bytes', marker='o', errorbar='sd')
        plt.title('Vazão da aplicação por concorrência')
        plt.tight_layout(); plt.savefig(out / 'throughput_vs_concurrency.png', dpi=180);
plt.close()
    plt.figure(figsize=(9, 5))
    sns.boxplot(data=df, x='operation', y='response_ms', hue='operation', legend=False)
```

```
    plt.title('Distribuição do tempo de resposta')
    plt.tight_layout(); plt.savefig(out / 'response_boxplot.png', dpi=180); plt.close()
    plt.figure(figsize=(9, 5))
    for op, part in df.groupby('operation'):
        x = np.sort(part['response_ms'].to_numpy())
        y = np.arange(1, len(x) + 1) / len(x)
        plt.plot(x, y, label=op)
    plt.xlabel('Tempo de resposta (ms)'); plt.ylabel('CDF'); plt.legend();
plt.tight_layout()
    plt.savefig(out / 'response_cdf.png', dpi=180); plt.close()
    print(summary.to_string(index=False))


if __name__ == '__main__':
    main()
```

A.7 Comandos de compilação e execução

```
mkdir -p src analysis server-data client-data
# salve os arquivos deste apêndice nos caminhos indicados
make
mkdir -p server-data client-data
./server --bind 0.0.0.0 --port 9000 --root ./server-data
# em outro terminal/nó:
./client 10.0.3.2 9000 PUT arquivo.bin arquivo.bin
./client 10.0.3.2 9000 GET arquivo.bin baixado.bin
sha256sum arquivo.bin baixado.bin
```

Adaptações necessárias para o experimento: no nó CORE, substitua o IP pelo endereço do servidor; salve a saída do cliente no CSV exigido pelo planejamento; execute vários clientes em paralelo; e mantenha os arquivos brutos, logs e parâmetros de cada cenário. Para alta concorrência, gere nomes remotos distintos ou declare explicitamente que os clientes testam sobrescrita concorrente do mesmo nome.

Referências oficiais

[1] Microsoft. “Install WSL”. https://learn.microsoft.com/en-us/windows/wsl/install

[2] CORE Network Emulator. “Installation” e “Install Docker”. https://coreemu.github.io/core/install.html ; https://coreemu.github.io/core/install_docker.html

[3] Docker. “Docker Desktop WSL 2 backend on Windows”. https://docs.docker.com/desktop/features/wsl/

[4] HashiCorp. “Vagrant VirtualBox provider”. https://developer.hashicorp.com/vagrant/docs/providers/virtualbox

[5] Enunciado fornecido. “Trabalho 1: Avaliação de Desempenho de Cliente-Servidor de Arquivos”, ICT/UNIFESP, 22/10/2026.